

AI 面试时代

你到底该拿什么，证明自己

当「会用 AI」人人都会写，面试桌上真正值钱的，是别的东西。



Robert 十年一线研发 · 做过面试官 · 每天在用 AI 写开源项目

一小时，聊点实在的。

开场 · 你我都有焦虑

四个问题，装在每个程序员心里

“

简历写「熟练使用 AI」

现在人人都这么写，写了约等于没写。

“

我明明用了 AI

怎么面试官问几个细节，就露馅了？

“

AI 帮我写的代码

算不算我的能力？讲多了心虚，讲少了吃亏。

“

平时零散用 AI

攒不成一个像样的项目故事。

这四个问题背后是同一件事：游戏规则变了，很多人还在用旧打法。

面试官考察的东西，正在从
「你会不会 **执行**」 变成 「你有没有 **判断**」

执行，AI 能替你做。判断，不能。

这条线贯穿今天全场：考察点变化、简历怎么写、追问逻辑、你该准备什么，都是它的延伸。

AI 岗位的热度，是真金白银的

+200%

2025 Q1 AI 领域求职人数
同比增幅（智联招聘）

400 万

2030 中国 AI 人才缺口预测
（麦肯锡）

+69.6%

AI 工程师岗位
求职热度增幅

缺口不在算法研究，在工程落地。 麦肯锡那 400 万缺口，大量分布在 AI 工程化、系统集成环节——也就是「把 AI 做成能上线的系统」这件事。这恰恰是有工程功底的人的机会。

面试官在考什么：哪里变了

过去 · 考执行

- 手写算法、数据结构
- 背 API、语法细节
- 从零实现某个功能
- 比谁写得快、写得多

现在 · 考判断

- 判断 AI 给的对不对
- 能不能快速定位问题
- 权衡与技术选型的理由
- 能不能对结果负责

变的是重心：从「生产代码」上移到「判断代码」。默认你会用 AI，考的是 AI 之上的东西。

考察点 · 不变

但有些东西，从没变，甚至更重要

计算机基础

操作系统、网络、数据库，不会因为 AI 而过时

系统理解

知道一个系统怎么跑起来、瓶颈在哪

Debug 能力

定位问题的能力，是 AI 时代最硬的通货

把问题讲清楚

能讲清判断和权衡，比会用工具值钱得多

工具变了，内功没变。AI 只会放大「有内功」和「没内功」的人之间的差距。

硬证据 · ANTHROPIC 随机对照实验

面试官的直觉，有实验撑腰

52 名工程师学同一个新库，一组用 AI、一组手写，做完考试：

50 分

用 AI 组
测验平均分

VS

67 分

手写组
测验平均分

差距最大的题型，正是 Debugging。 「判断代码错没错、为什么错」这个能力，最容易因为依赖 AI 而萎缩。而面试官追问细节、让你现场找 bug，本质就是在复现这个实验：验证你是真懂，还是 AI 帮你写的。

「熟练使用 AI」为什么正在失效

AI 代劳型

让 AI 写完，拿了就走，不回头理解。完成最快、几乎不犯错——但什么都没掌握。

掏空了自己

先生成后理解型

让 AI 写完，回头追问：为什么这么写？改成 X 会怎样？用 AI 检查了自己的理解。

保住了能力

两种操作几乎一模一样，结果天差地别。「熟练使用 AI」之所以失效，是因为它证明不了你是哪一种。

字节三面，一道意料之外的题

“

你日常开发用 Claude Code 还是 Copilot？为什么选它？

不是考算法，不是考系统设计，考的是——你的工具选择背后，有没有判断。

- **答不出「为什么选」** 说明你只是随手在用，没有对工具做过判断和比较。
- **这在 2026 已是面试标配题** 面试官用它快速区分「随手用」和「有判断地用」。

真实面经 · 案例二

一个「只会调 API」的人，怎么当场哑口

候选人简历写：调用过大模型 API，做过 RAG 服务。面试官三连问：

Q1 你的 RAG 服务，每秒能处理多少请求？

Q2 检索速度怎么提上去？

Q3 要是向量数据库崩了，你怎么应对？

结果

连最基础的兜底都答不上。简历只写「调用过 API」，直接被刷。

公司招的不是「会调接口的人」，是「能解决问题、能对可用性负责的人」。

面试官到底在验证什么

追问，本质是一场「可验证性测试」。

能被你当场解释、推演、debug 的

→ 是你的真能力

只能背出结论、一深问就崩的

→ 是 AI 的能力，借你的嘴说出来

追问不是刁难，是在剥离「AI 的功劳」和「你的功劳」。

面试官一层一层往下逼

第一层

是什么

你做了什么？——大部分人都过得了。

第二层

为什么

为什么这么选？权衡了什么？——「代劳型」开始卡。

第三层

如果变了会怎样

改需求、加十倍压力、出故障——只有真懂系统的人答得上。

面试官就是靠这三层，把你的理解深度探到底。

好的回答，长什么样

“

做知识库问答，一开始用 CoT，复杂问题分解不好；换成 ReAct，让模型边想边调工具检索，**准确率提升了 15%**。ToT 我也试过，效果好但 token 消耗三倍，**线上成本扛不住**，最后只在离线深度推理时用。

为什么这个回答能拿 offer

- **说清了 trade-off:** 效果提升多少、成本增加多少，都有数字。
- **说清了为什么这么选:** 这比背一堆概念，强一百倍。

大厂在考什么 · 真实 JD 横切

四家公司的岗位，指向同一件事

DeepSeek

专设「Agent Harness」团队，对标 Claude Code，大规模抢人

字节跳动

多智能体框架研发，Agent 在审核系统的落地与创新

蚂蚁

PaaS 智能体平台研发专家，要求多 Agent 编排

美团

大模型应用后端工程师，Agent 测评标准输出经验

共性：要的是「能把 Agent 做落地」的人，不是「只会调接口」的人。

一份 JD 拆到底 · DEEPSEEK AGENT HARNESS

它到底要什么样的人

- 1 重度用 Claude Code / Cursor, 且懂其内部机制
- 2 能搭 Agent 运行时、容器服务、轨迹回放与调试
- 3 Docker / K8s / CI/CD / Git —— 明确硬要求, 非加分
- 4 理解 Context Window 与 Token 机制
- 5 MCP / Tool Use / Function Calling (加分项)

翻译成一句话: 它要的不是「会用 AI 的人」, 是「能把 AI 做成可靠系统、能对结果负责的人」。

认清赛道 · 扒了 200+ 份 JD

「Agent 工程师」不是一个岗，是一类岗

应用开发

Infra 平台

算法

评测

产品 PM

多模态

- **LangChain 仍是第一框架** 但 LangGraph + MCP 已是生产标配，89% 团队做 Observability。
- **堆一排框架名，是 HR 抄模板的表现** LangChain+LlamaIndex+AutoGen+Dify+Coze... 堆得越多越可疑。
- **简历别堆框架** 要堆「为什么选这个框架」的判断。

2026 Agent 岗，真正要点的技能树

工程底座（硬要求）

Docker · K8s · CI/CD · Git 工作流——几乎每份工程岗 JD 的地基

Agent 核心机制

ReAct 循环 · Function Calling / Tool Use · MCP（生产标配）· 上下文与 Token · 记忆管理

系统能力（后端直接复用）

API 设计 REST/gRPC · 微服务 · 消息队列 · 向量库 · 高并发与成本控制

可观测与评估

89% 生产团队做 Observability · LLM-as-a-Judge 评测体系

框架名（LangChain / Dify...）是「术」，会换；把 Agent 做成可靠系统的能力是「道」，不过时。

为什么 Debug 是 AI 时代最硬的通货

AI 能写出「看起来对」的代码，
但「为什么错、错在哪」是判断，不是生成。

不会 debug 的人

面对 AI 写的代码，只能当橡皮图章——它说对就对。

会 debug 的人

是 AI 时代的质检长——AI 生产，你把关，你负责。

反直觉真相 · 给年轻人

卡壳，本身就是学习发生的地方

那个实验还有一个发现：手写组遇到了更多错误——

而正是通过独立解决这些错误，
他们练出了 debug 能力。

AI 最危险的地方，不是写错，是太顺。 它把「卡壳」这件事抹平了，于是也顺手抹掉了藏在卡壳里的成长。想在面试桌上有东西，平时就得刻意留一点「自己先卡一会儿」的权利。

两极分化，中间的人最缺

纯算法岗

卷学历、卷论文；离业务落地远，门槛越来越高。

复合型人才

懂工程落地的 AI 应用工程师 / 架构师。
既懂工程化，又懂 AI，能把模型接进业务。最大缺口。

纯后端岗

竞争白热化，薪资增长乏力，初级可替代性强。

中间那个「既懂工程落地、又懂 AI」的复合型人才，是 2026 企业争抢的核心——这是转型的黄金窗口。

转岗 · 后端工程师

后端的经验，不是包袱，是护城河

大模型服务，本质就是一类新型分布式系统。你的老本事直接迁移：

高并发 / 微服务

Agent 服务的负载波动、峰值请求，你最懂

成本控制

推理成本控制 = 有限资源下性能最大化，你的老本行

高可用 / 降级熔断

服务宕机怎么保 99.9%，正是企业付费的核心

面试真题：「大模型服务宕机怎么保 99.9%？调用暴涨 10 倍怎么控成本？」——这道题，是「API Caller」和「AI 架构师」的分水岭，也正是后端最擅长的。

转岗 · 前端 / 全栈

前端的「交互 + 流程」，正是 Agent 最缺的

你熟 状态管理

→ 对应 Agent 的状态机 / 记忆管理

你熟 异步 / 流

→ 对应 Agent 的流式输出 / 工具调用

你熟 接口与 Schema

→ 对应 Agent 的 Function Calling / 结构化输出

你熟 用户流程

→ 对应 Agent 的任务规划 / 多步执行

Agent 最缺的不是「会聊天」，是「交互 + 流程 + 可靠性」这些工程能力——前端很有优势。

转岗最常踩的三个坑



死磕底层算法

一上来啃 PyTorch、反向传播、GPU 集群——那是研究员的活，做 AI 应用根本用不上。就像用 MySQL 不必懂 B+ 树。



只刷教程不做实战

收藏几十 G 资料，却连一个 RAG 知识库都搭不出来。企业要「能落地」，不要「会背理论」。



简历只写「调用过 API」

最常见的坑，直接被刷。公司要的是能设计反馈机制、能监控恢复、能对结果负责的人。

方法 · 可套用的表达公式

AI 经验的正确讲法

别写「熟练使用 Claude Code」。用这个结构讲：

1 我用 AI 做了什么

执行

2 我做了哪些判断

选型 · 权衡 · 否决了什么

3 我怎么验证它是对的

测试 · review · 发现并修了什么

4 结果和我的责任

我为这个结果负什么责

公式的核心：把你的「判断」和「负责」讲出来，而不是把「会用工具」讲出来。

方法 · 同一段经历，两种讲法

把订单服务这件事，讲给面试官听

弱讲法

「我用 Claude Code 写了一个高并发订单服务。」

只证明了：会用工具。

强讲法

「核心逻辑我让 AI 生成，但并发模型是我定的；AI 第一版用了乐观锁，**我判断在这个写多读少的场景会大量冲突，改成了分段锁**；上线前我压测，**发现一个连接泄漏，定位是连接池没复用**。」

证明了：判断力 + 验证力 + 对结果负责。

行动 · 你现在就该做的

2026，从今天开始做这五件事

→ 把 AI 当老师，不当代笔——生成完，回头追问「为什么」

→ 刻意保留卡壳——先自己啃一会儿，再问 AI

→ 每个项目，都能讲得出「我的判断在哪」

→ 往深水区扎根——系统设计、分布式、debug、合规安全

→ 攒一个能扛住「三层追问」的真实项目

长期来看

什么才是 AI 替不走的

不是「你会不会用 AI」，
是「你能不能对结果负责」。

AI 把执行做到趋近免费，判断反而成了唯一稀缺的东西。

面试桌上真正拿得出手的，不是「我会用 AI」，
是「我在用 AI 的时候，没有掏空自己」。

那么问题来了

讲了这么多——

判断力、能扛追问的真实项目、往深水区扎根.....

这些东西，到底怎么练出来？

平时零散用 AI，攒不出来。这正是我们训练营想解决的事。

培养把 Agent 做成可靠生产系统的人

一条主线，四个递进的载体——练的不是 Demo，是能上生产、能写进简历的真项目：



方法论三支柱

SDD 规范驱动（想清楚） **Harness Engineering**（建好环境） **Loop Engineering**（让 Agent 自主循环）

11 周，从需求分析、方案设计，一路走到集群上线、开源贡献。

你能带走什么

这门课，给你面试桌上真正拿得出手的东西

- 四段能扛住「三层追问」的真实大型项目经验
- 一套讲得清「判断」和「负责」的表达方式
- 被 DeepSeek 这类大厂明码标价的 Harness 能力
- 一段真实开源 PR 经历，GitHub 上查得到的代码
- 最重要的——用 AI，但没有掏空自己的成长方式

工具会过时，判断会留下。

一个小时，希望给你的不只是面试技巧，是未来很多年都拿得出手的东西。